Bangla Speech-To-Braille Interaction Device for Visual and Hearing Impaired



Rafia Rizwana Rahim, Atik Ishrak Al Nahian, Rakiba Benta Kalam, Atanu Sen Gupta, and Nusrat Binta Nizam

Abstract This research project proposes a prototype design for a refreshable braille display to convert Bangla Speech to Braille output. It aims to be a communication and learning module for deaf-blind or blind people to pursue inclusive education. First, we took speech input from a transducer, and speech recognition was done by DeepSpeech, a Natural Language Processing framework developed by Mozilla. Then, the converted text output was linked with its corresponding braille pins and was shown in a refreshable braille display. Finally, the entire computation was carried away by a Raspberry Pi. We have proposed the usage of our prototype as a special ICT device in the journey towards inclusive education in Bangladesh. In addition, future development has been suggested to broaden the periphery of the project.

Keywords DeepSpeech · Bangla natural language processing · Refreshable Braille display

1 Introduction

The goal of this research project is to create a refreshable braille display that accepts Bangla speech as input and outputs active braille pins. It is primarily intended to serve as a communication and learning tool for deaf-blind persons, particularly those aged 5–18. When we saw that there were few technologies for blind students and none for deaf-blind pupils, we were inspired to create such a product. There are 285 million visually impaired (VI) people in the world [1], where 90% of them are from developing countries [1]. Nearly 19 million children below 15 age group are VI [1].

R. R. Rahim · A. I. Al Nahian · R. B. Kalam · A. S. Gupta
Department of Mechanical Engineering, Bangladesh University of Engineering and Technology, Dhaka 1205, Bangladesh
e-mail: 1710045@me.buet.ac.bd

N. B. Nizam (✉)
Department of Biomedical Engineering, Bangladesh University of Engineering and Technology, Dhaka 1205, Bangladesh
e-mail: nusratbintanizam@bme.buet.ac.bd

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2022

S. Hossain et al. (eds.), *Proceedings of International Conference on Fourth Industrial Revolution and Beyond 2021*, Lecture Notes in Networks and Systems 437,
https://doi.org/10.1007/978-981-19-2445-3_47

Among them, 90% children do not have access of education in developing countries [2]. Bangladesh is a developing country which has 341,819 VI children in the 6–11 age group [5]. Also, a significant portion is illiterate. The literacy rate is 57.7%, which demands ensuring inclusive education, where the system will make education affordable for all, irrespective of their background or disabilities [6]. Implementation of special Information and Communication Technology (ICT) devices in education can help to develop the quality of life and empower the VI and deaf-blind people [7]. But the accessibility and availability of ICT educational tools are a major issue because these tools can help to ensure equal opportunities for all students across the world.

2 Literature Review

Even while Braille Displays are pretty famous, adding a language-specific speech to the braille module is the real issue. It is a pity that Bangla, the world's seventh most spoken language, lacks a fully developed speech-to-text module that we may use in our project. We found the closest potential work that can assist us in creating our voice recognition module when searching for a good speech recognition approach. We found the following suggested approaches and theoretical results for Bangla Speech to Text Conversion Linear Predictive Coding (LPC) and Cepstral Coefficients for feature extraction and a Multilayer Perceptron (MLP) approach for recognition [9], Mel Frequency Cepstrum [10], Unsuper vised learning by recognizing repeated patterns [11], tokenizing the input speech [12], unsupervised term discovery to cluster repeated patterns [13], classification of unlabeled data [14]. More recent studies used CNN, RNN, and LSTM-based models for Bangla speech recognition purpose [17]. Also, augmented dataset has given closer mnemonics and better results than other studies [18]. Hannun et al. [15] showed a deep recurrent neural network architecture to recognize continuous speech. Experimentation showed that it gave highly accurate Transcriptions to continuous speech. With data of 600hours, the model yields 90% accuracy for Mandarin Chinese and English. They published as Mozilla's open-source framework DeepSpeech, which can be used to model any language for speech recognition and, therefore, has been used in our research. To support the requirements of DeepSpeech, we searched for speech to text dataset and found that the dataset provided by the openslr platform is the most suitable for our purpose [19]. Previous studies have also been done using the same dataset. For Example Sultana et al. (2012) used the corresponding dataset in their SAPI-based speech recognition module [16]. Therefore, this dataset is a proven one to be used in our research.

3 Dataset Preparation

A huge preprocessed dataset is the primary need of any deep learning frame work. DeepSpeech also requires over 100,000 audio and text samples and numerous other auxiliary files to begin its training. The steps of dataset preprocessing can be described sequentially.

3.1 Dataset Selection and Download

We looked for an open-source Bangla audio dataset and found 250 thousand audio of 347 h and corresponding text data in the openslr platform [19], where it contained 229 h of unique speech to text data and the rest of the dataset was either repeated or augmented. The dataset was in a very raw format and needed preprocessing.

3.2 .wav Conversion, Riff Code Addition and Audio Data Debug

DeepSpeech only accepts.wav audio files with a specified riff code attached before the audio file. Using AudioConverter 0.3.0 [22], we converted the .flac files to.wav files with riff code attached. Then we built ftwdebug.py [21], a debugger program to see if any audio files were missing the riff code. Furthermore, DeepSpeech will not accept audio input with a duration of fewer than 3 s. As a result, we built a new debugger code [21] to erase and exclude all audio with a duration of fewer than 3 s.

3.3 Corresponding Text Manual Correction and Unicode Debug

DeepSpeech requires audio files to be accompanied by a text file that contains no numerical characters or punctuation marks. As a result, all of these had to be manually erased. The openslr dataset included a .tsv file containing file names (without extensions), anonymous ids, and matching text for each audio file. As a result, we initially copied it into Excel. Next, we divided the spreadsheet file into 250 separate excel files, each with 1000 items, to make human corrections easier. All punctuation marks and emoji were then manually removed. In this case, the number of characters was replaced with Bangla text. Unfortunately, the text contained an unexplained error, and DeepSpeech could not locate Unicode for the Bangla script. So we built xldebug.py [21], a debugging program to see if each of our characters has a UTF+8 code associated with it.

3.4 .csv File Generation

DeepSpeech requires the following format for audio text in.csv: filename (with extension), file size (in Kilobytes), text. To create such a .csv file, we created the file name size.py script [21], which can create .csv files containing the file name and size. Our audio files were divided into 14 folders, each with 14 subfolders. We put all audio files in one folder and made a .csv file with the file name and size.

3.5 Text Input In .csv Files

Now, we had two files one having the file name and corresponding texts, the other having file name and size. We needed to merge these two excel sheets concerning file names. When we did so, we saw that several audio files did not have written text with them. Also, some of the audio files did not exist in our dataset, but their file name and text were written in the excel file. So we needed to delete those entries manually. Here is our sequential operation we copied the file names in a text file for each segmented excel file. Then we run a code File Copy Reading txt to Folder and Check.py [21] that will copy the file reading the .txt from our audio folder. It will also show which files do not exist. So we can remove those entries manually. This same process had to be repeated 250 times. Then we can merge text with file name and size .csv file.

3.6 Training, Development, and Testing Dataset

DeepSpeech requires three folders having audio files and .csv files containing the file name, size, and text. They are called training (train), development (dev), test folders. In each training cycle, train, dev, and test contain 70, 20, and 10% audio data. For example, we trained our model 5 times, each with 40–80 thousand data. For this, we created test, train, dev folders for each training using the above-stated manner.

3.7 Generating alphabet.txt File

We needed a code to generate all alphabet of Bangla in a text file. Moreover, we needed it to tell the training framework that we are using a different alphabet set than English, and it is Unicode based. So, we created a python code alphabet.py [21] that generates an alphabet of Bengali language and saves it in a text file.

3.8 Generating a Scorer File

A scorer is a language model, and DeepSpeech uses it to improve the accuracy of the transcription. A language model predicts which words are more likely to come after one another. The scorer determines the likelihood of words appearing together. DeepSpeech's default scorer is trained using the LibriSpeech dataset. The LibriSpeech dataset is based on LibriVox, an open repository of public domain and out-of-copyright works. Therefore, we needed scorer training DeepSpeech in another language, training a speech recognition model for a particular domain. In our case, it is day-to-day formal Bangla speeches mostly available in news reports and to improve the accuracy of the transcription.

DeepSpeech supports the optional use of an external scorer. We collected many of the sentences from Bangla Wikipedia through a module named banglaWIKI corpus, which allowed us to scrap through Wikipedia and download the Bengali sentences, which we will use to build the scorer. It supports our DeepSpeech model with predefined sentences that we regularly use. If a model does not understand a voice fully, it tries to match its word with the scorer file and shows outputs accordingly.

4 Design and Methodology

4.1 Bangla Automatic Speech Recognition

A microphone should capture the sound waves, convert them to a digital signal, and store them in memory. The sound must be linked to the corresponding test. To find the appropriate text for the sound, we need pre-requisites of 2 types a large dataset with a variety of audio and text and a framework having an Acoustic Model, Pronunciation Model, and Language Model (Fig. 1).

Preparation for DeepSpeech Framework We have used Google's cloud-based machine learning platform, Google Colab, to train our model. To start training in Google Colab, first, we need to set up an environment of DeepSpeech in the following manner. Installation of DeepSpeech, mounting Google Colab with Google Drive to train the data from the drive directly, enabling GPU, and installing other packages such as an appropriate version of TensorFlow have been done, respectively. After preparing the train and dev folder, we need to start the training by calling flags on DeepSpeech. Our ultimate target is to decrease the value of the loss. The validation part is the development part using the dev folder.

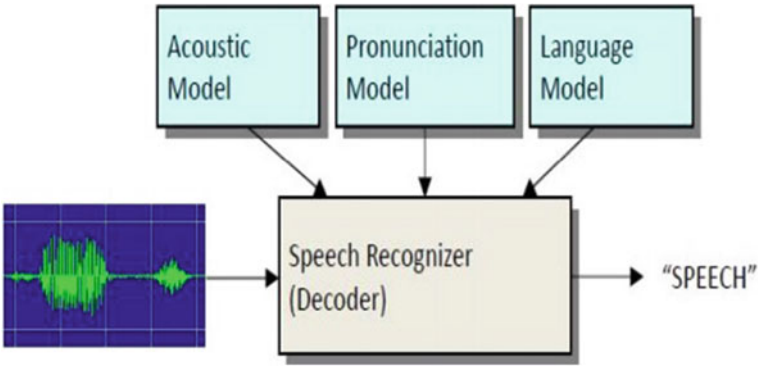


Fig. 1 Speech recognition flow chart

4.2 Bangla Text to Braille (TTB) Conversion

Every character of the Bangla language has a unique Unicode. For every Unicode, a character has a unique combination of braille pins. Braille is a language where letters are shown via the combination of 6 pins in a braille cell numbered vertically. Most of the letters in Bangla need one set of braille pins to resemble the character. Nevertheless, prefixes and suffixes are incorporated when it comes down to words, joint letters, or sentences. We have followed all the rules of Bangla Braille to Convert Bangla Text to Braille pins. The following rules were considered while setting conditional loops in the TTB code, btb.py. Our code (btb.py) [21] first splits every word in the text obtained from DeepSpeech. If there is any joint letter, it is detected by Unicode of hosonto between those letters. Then the braille pins are detected according to the rules of Bangla Braille Adaption [27]. The numbers of braille pins are the input of our refreshable braille display. The display is designed so that the output is shown cell by cell, one cell at an instant (Figs. 2, 3 and 4).

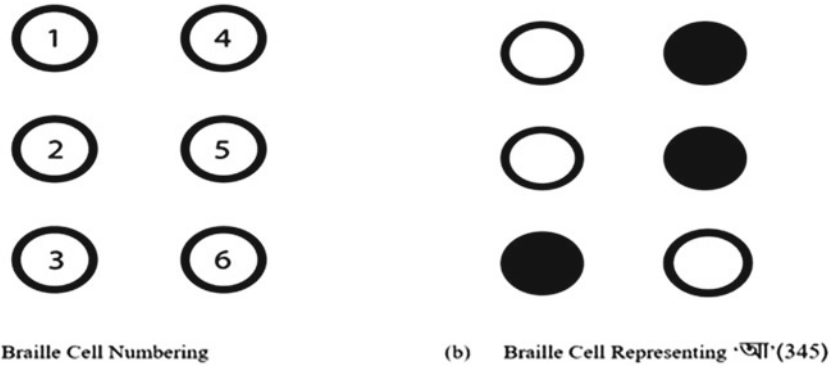


Fig. 2 Braille cell representations

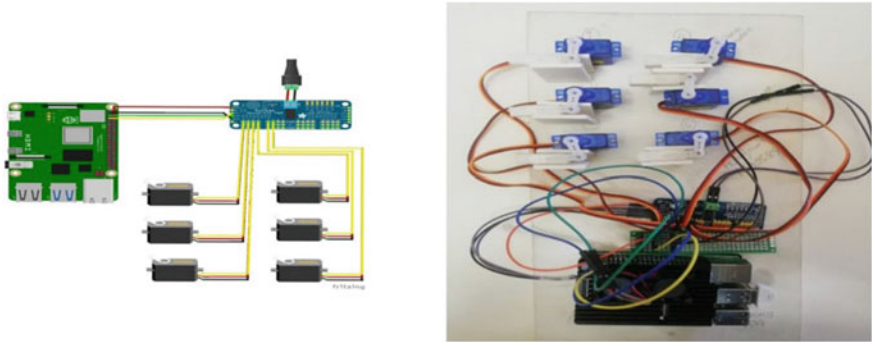


Fig. 3 Electrical circuit simulation using fritzing and electrical circuit

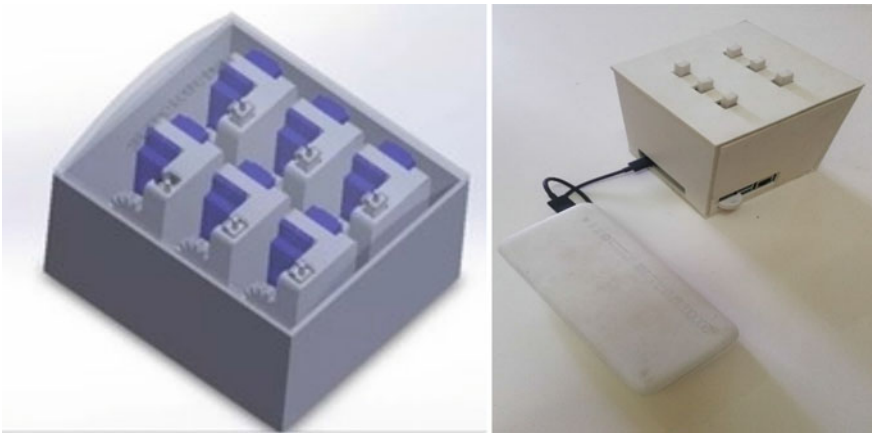


Fig. 4 Solid works design and final assembly

4.3 Refreshable Braille Display

Electrical Circuit Design: We have used six independent servo motors with a linear actuation system to show the braille pins obtained. Here the servo has been controlled by a Pulse Width Modulation (PWM) controller named Adafruit PCA9685. This PWM controller takes input from Raspberry Pi and sends corresponding signals to the servos. The servos' precise motion helps to step up the braille pin when necessary. We have done our circuit design using Fritzing [24]. Once our circuit was complete, we combined all of our programs into a single file called Speech to Braille Hardware.py. [21] While operating, this program requests the user's input audio stream, transforms it to text, and then generates the matching Braille pin numbers and servos. The electrical simulation was also used to create a printed circuit board (PCB).

Mechanical Part Design: We intend to use a rack and pinion gear system as our primary hardware and designed the device accordingly in SolidWorks [25]. However, because of the lockout in the Covid-19 issue, we were unable to 3D print our rack and pinions. As a result, we devised a less expensive alternative: the reverse crank mechanism. The servo shaft will be attached to a crank here. The connecting rod will be linked to the other end of the crank. The connecting rod is connected to a piston (braille pin) that can reciprocate with a predetermined slot. As a result, the reverse crank mechanism acts as a linear actuator, converting the servo's rotating motion to the linear motion of the braille pin [26].

After trailing for several weeks, we saw that our prototype was working fine. So we did our final assembly. Here the circuit is mounted in a 10cm*10cm*10cm box. The microphone adapter is directed outward. The raspberry pi is getting power from a power bank.

5 Results

Here src means the actual source text of the speech, and res means predicted text by DeepSpeech. There are some critical features of our testing data (Fig. 5). First, the word count per sentence and letter count per word is almost ac- curate. Second, common verbs and words are predicted correctly. Finally, the mnemonics of the words were predicted correctly. That is, the position of vowel and consonant modifiers and silent letters are in the correct position. Bangla is segmented into groups of letters according to the way they sound. Unfortunately, our model could not predict the exact sound, and the place is taken over by the closed possible sound in that group. Here WER means Word error Rate, and CER means character Error Rate. In our best WER, we can see that our model recognized 77% of characters in a sentence and

```
WER: 0.333333, CER: 0.133333, loss: 15.634764
 - wav: file://../drive/MyDrive/Data1/test/195590bd5b.wav
 - src: "অনেক দামের সুতা"
 - res: "সনের দামের সুতা"
--------------------------------------------------------
WER: 0.500000, CER: 0.384615, loss: 23.318504
 - wav: file://../drive/MyDrive/Data1/test/19e034d0a1.wav
 - src: "ইউনিবিজয় নামে"
 - res: "সমিনিজল নামে"
--------------------------------------------------------
WER: 0.500000, CER: 0.263158, loss: 21.237675
 - wav: file://../drive/MyDrive/Data1/test/199c157a10.wav
 - src: "হত্যা করা হয়েছে বলে"
 - res: "সতা করা হরিছে বলে"
```

Fig. 5 Example of ASR test result

67% of the word in a sentence. So as we can see from Figs. 6 and 7, we have trained our model, but we still have a lot of WER and CER errors. Therefore, to overcome those errors, we need to improve our language model.

Our TTB code gives 100% accurate results while converting text to braille (Figs. 8 and 9).

```
Best WER:
--------------------------------------------------------------------------------
WER: 0.333333, CER: 0.230769, loss: 10.730057
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/918e8af019.wav
 - src: "দিয়ে দিত বলে"
 - res: "সি দিত বলে"
--------------------------------------------------------------------------------
WER: 0.500000, CER: 0.480000, loss: 56.481258
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/915a7dd10c.wav
 - src: "অর্থমন্ত্রণালয়কে জানিয়েছে"
 - res: "সতনাজবাকে জানিয়েছে"
--------------------------------------------------------------------------------
WER: 0.500000, CER: 0.388889, loss: 25.654314
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/91370868fc.wav
 - src: "সমাজের পূর্বাজিত"
 - res: "সমাজের পমাধিত"
--------------------------------------------------------------------------------
WER: 0.500000, CER: 0.157895, loss: 20.535526
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/90cd3a27bc.wav
 - src: "আমলা না করার জন্য"
 - res: "সাবলা না করার জন্দল"
--------------------------------------------------------------------------------
WER: 0.500000, CER: 0.555556, loss: 18.539881
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/91a97a499d.wav
 - src: "আশ্রয় দিল"
 - res: "স দিল"
```

Fig. 6 Best case scenario test result

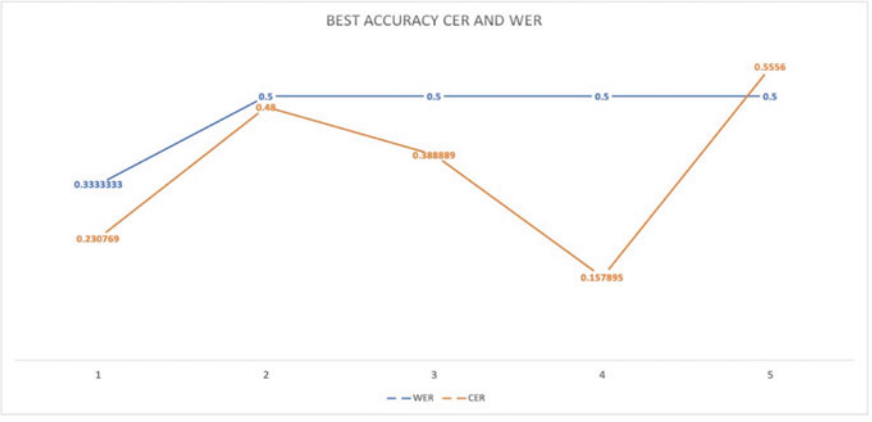


Fig. 7 Accuracy of CER and WER for best case scenario

```
Worst WER:
--------------------------------------------------------------------------------
WER: 2.000000, CER: 0.545455, loss: 16.047106
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/90a5ce2ba8.wav
 - src: "বেচারামবাবু"
 - res: "সিচানাম ববু"
--------------------------------------------------------------------------------
WER: 2.000000, CER: 1.250000, loss: 13.515151
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/90cd3119a6.wav
 - src: "কর্ণ"
 - res: "সন্ল স"
--------------------------------------------------------------------------------
WER: 2.000000, CER: 0.333333, loss: 9.219383
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/91358a8456.wav
 - src: "তারপরও"
 - res: "সার পরও"
--------------------------------------------------------------------------------
WER: 3.000000, CER: 1.000000, loss: 21.332855
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/914cc6a1c5.wav
 - src: "কানাইঘাট"
 - res: "সনাই দা তান"
--------------------------------------------------------------------------------
WER: 3.000000, CER: 0.750000, loss: 13.563680
 - wav: file://../drive/Shareddrives/SynthiaSoft.com-Unlimited-03/Data_for_Atik/Data_for_Atik_4/testwav/91867b3a36.wav
 - src: "পলাশবাড়ী"
 - res: "সনা শ বারি"
```

Fig. 8 Worst case scenario test result

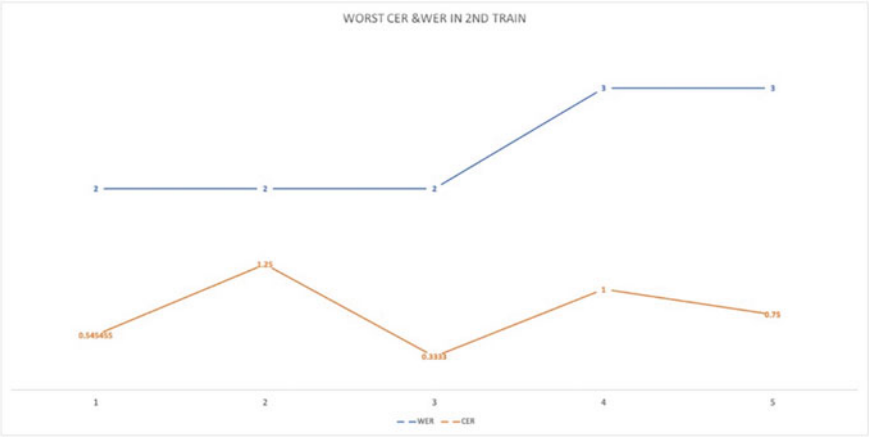


Fig. 9 Accuracy of CER and WER for worst case scenario

6 Discussion

The ASR results described in the figures are self-explanatory to understand how our ASR model is dealing with test data. The model can successfully identify mnemonics of common verbs, vowels, consonant, and vowel modifiers. More data can reduce the loss in this model and therefore give a better accuracy. We have realized that perfecting any speech recognition module could take months. Furthermore, it is a one-way communication medium; a person may convert speech to braille but not vice versa using the same instruments. How- ever, we do not consider all of this

a weakness; instead, we believe that it will only open the doors to a plethora of opportunities soon. It is self-evident that the accuracy of any ASR model improves as the dataset size grows. We used a dataset that was created by chopping up news snippets for our project. As a result, it is unable to distinguish the dialect of a specific Bangladeshi district. Its vocabulary has to be expanded in order to recognize more complex works. As a result, the production of datasets is our key priority as we upgrade our prototype. Our prototype is a one-way communication medium. We intend to incorporate Braille to Speech using the same devices. So when a deaf-blind person is asked something using our device, he/she can get an immediate response. Such de- vices can work as their eyes and voice. This complete package can help millions of people to demolish the barriers around them and add them to mainstream education.

7 Conclusion

Our project has a broad spectrum of usage when it comes down to inclusive education. It can be used as a communication and learning device not only for deaf-blind people but also for blind people. Our equipment is simple to use for school-aged youngsters who are deaf-blind. Because it records audio and subsequently translates it to braille, the gadget can be used as a note-taking device for them and a great aid while listening to lessons. As a result, implementing our initiative following the Bangladesh Education Ministry's objective can illuminate the path to inclusive education on its own.

Acknowledgements At first, we would like to thank our family members, especially our parents. Without their hospitality, the project would not be possible in the lockdown due to the Covid-19 situation. Next, we are ever grateful to Satyaki Banik and Nusrat Binte Nizam, who greatly encouraged us to take up a project to bridge the gap between Bangla Natural Language Processing (NLP) and Mechanical Engineering. We want to thank Apurba Sarker for his initial involvement in preparing DeepSpeech Framework, Rasman Mubtasim Swargo, and Aniruddha Ganguli for their knowledgeable suggestions regarding NLP. Finally, we are grateful to Partha Pratim Das for the Google Colab Pro Support and Shakti Banik for his contribution in conceptualizing the design.

References

1. WHO Media Centre. Visual impairment and blindness. http://www.who.int/mediacentre/factsheets/fs282/en/ (2013). Accessed 15 March 2014
2. United Nations. Factsheet on Persons with Disabilities. http://www.un.org/disabilities/default.asp?id=18 (2013). Accessed 25 May 2013
3. Mitra, S., Posarac, A., Vick, B.: Disability and poverty in developing countries: a snapshsot from the World Health Survey. (World Bank Sp discussion paper (1109)), The World Bank, Washington, DC, USA (2011)
4. Rahman, K.F.: Blindness, 'Vision 2020 and Bangladesh. Finan. Express **20**(436), 10 (2012)

5. Das, A.: Inclusion of student with disabilities in mainstream primary education of Bangladesh. J. Int. Dev. Cooper. **17**(2), 1–10 (2011)
6. EFA Global Monitoring Report 2010. Reaching the Marginalized. United Nations Educational, Scientific and Cultural Organization. Oxford University Press, Oxford, UK
7. Hirayama, M.J.: Digital talking book contents of beading handicraft design diagram for visually impaired persons. In: 4th International Conference on Interaction Sciences (ICIS), pp. 96–101, Busan, South Korea (2011)
8. CDD. https://cdd.org.bd/deaf-blind-support/
9. Paul, A.K., Das, D., Kamal, M.M.: Bangla speech recognition system using lpc and ann. In: 2009 Seventh International Conference on Advances in Pattern Recognition, pp. 171–174, February 2009.
10. Ahammad, K., Rahman, M.M.: Connected Bangla speech recognition using artificial neural network. Int. J. Comput. Appl. **149**(9), 38–41 (2016)
11. Park, A.S., Glass, J.R.: Unsupervised pattern discovery in speech. IEEE Trans. Audio Speech Language Process. **16**(1), 186–197 (2008)
12. Kamper, H., Jansen, A., Goldwater, S.: Unsupervised word segmentation and lexicon discovery using acoustic word embeddings. IEEE/ACM Trans. Audio Speech Language Process. (TASLP), **24**(4), 669–679 (2016)
13. Bansal, S., Kamper, H., Lopez, A., Goldwater, S.: Towards speech-to-text translation without speech recognition. arXiv preprint arXiv:1702.03856 (2017)
14. Honglak Lee, Peter Pham, Yan Largman, and Andrew Y. Ng. Unsupervised fea- ture learning for audio classification using convolutional deep belief networks. In Y. Bengio, D. Schuurmans, J. D. Lafferty, C. K. I. Williams, and A. Culotta, editors, Advances in Neural Information Processing Systems 22, pages 1096–1104. Curran Associates, Inc., 2009.
15. Hannun, A. et al.: Deep speech: scaling up end-to-end speech recognition. Baidu Research – Silicon Valley AI. arXiv:1412.5567v2. [cs.CL], 19 December 2014
16. Sultana, S., Akhand, M.A.H., Das, P.K., Rahman, M.H.: Bangla speech-to-text conversion using SAPI. In: 2012 International Conference on Computer and Communication Engineering (ICCCE), pp. 385–390 (2012)
17. Sabab, M., Chowdhury, M.A.R., Nirjhor, S.M., Uddin, J.: Bangla speech recognition using 1D-CNN and LSTM with different dimension reduction techniques. In: International Conference for Emerging Technologies in Computing, pp. 158–169. Springer, Cham (2020).
18. Maruf, M., Faruque, M., Muhtasim, M., Nelima, N.N., Mahmood, S., Riad, M., Uddin, M.: Comparative study of effective augmentation method for Bangla ASR using convolutional neural network." In: Emerging Technologies in Data Mining and Information Security, pp. 203–213. Springer, Singapore (2021)
19. Large Bengali ASR training data set. https://tinyurl.com/Open-SLR
20. Bangla Text to Braille code (2021). https://tinyurl.com/Bangla-Text2Brail
21. Shobdo-Kolpo-Drum/Project-codes (2021). https://github.com/Shobdo-Kolpo- Drum/Project-codes
22. AudioConverter 0.3.0. https://pypi.org/project/AudioConverter/
23. Mozilla/DeepSpeech. https://github.com/mozilla/DeepSpeech
24. Simulated circuit design.fzz (2021). https://tinyurl.com/Circuit-Design-A25
25. Shobdo Koplo Drum (2021). https://tinyurl.com/Shobdo-kolpo-drum
26. Group A25 Demo (2021). https://tinyurl.com/Group-A25
27. Hossain, S.A. et al.: Bangla braille adaptation. In: Technical Challenges and Design Issues in Bangla Language Processing, Chapter 2 (2013)